INT1339 - Ngôn ngữ Lập trình C++

Chồng Toán tử
Operator Overloading

Giảng viên: Lê Hà Thanh

lehathanh@ptithcm.edu.vn

Nội dung

- Các hàm toán tử
- Kiểu số phức
- Chuyển kiểu

Lớp số phức complex

```
class complex {                    // very simplified complex
        double re, im;
public:
        complex(double r, double i) :re{r}, im{i} { }
        complex operator+(complex);
        complex operator*(complex);
};
```

Các toán tử có thể được khai báo/định nghĩa lại – Chồng toán tử

Ý nghĩa quy định (định nghĩa) các toán tử sau có thể được khai báo:

+	-	*	/	%	^	&
|	~	!	=	<	>	+=
-=	*=	/=	%=	^=	&=	|=
<<	>>	>>=	<<=	==	!=	<=
>=	&&	||	++	--	->*	,
->	[]	()	new	new[]	delete	delete[]

- Khai báo hàm toán tử: từ khóa operator sau đó là toán tử

operator<<

Các toán tử không thể khai báo chồng

Các toán tử sau không thể định nghĩa lại:

:: toán tử phạm vi

. toán tử chọn thành viên

.* toán tử chọn thành viên thông qua con trỏ tới thành viên

sizeof kích thước đối tượng

alignof hàm căn chỉnh đối tượng

typeid thông tin kiểu **type-info** của một đối tượng

? : toán tử đánh giá điều kiện

Cài đặt và sử dụng hàm toán tử

- Cài đặt hàm như bình thường
- Chú ý ngữ nghĩa của toán tử để viết xử lý nội hàm

- Sử dụng:

```
void f(complex a, complex b)
{
        complex c = a + b;              // shorthand
        complex d = a.operator+(b);     // explicit call
}
```

Toán tử nhị phân (Binary operator)

```
class X {
public:
        void operator+(int);
        X(int);
};

void operator+(X,X);
void operator+(X,double);

void f(X a)
{
        a+1;        // a.operator+(1)
        1+a;        // ::operator+(X(1),a)
        a+1.0;      // ::operator+(a,1.0)
}
```

Toán tử đơn phân (Unary operator)

```
class X {
public:          // members (with implicit this pointer):

    X* operator&();          // prefix unary & (address of)
    X operator&(X);          // binary & (and)
    X operator++(int);       // postfix increment
    X operator&(X,X);        // error: ternary
    X operator/();           // error: unary /
};

// nonmember functions :

X operator–(X);              // prefix unary minus
X operator–(X,X);            // binary minus
X operator––(X&,int);        // postfix decrement
X operator–();               // error: no operand
X operator–(X,X,X);          // error: ternary
X operator%(X);              // error: unary %
```

Toán tử và Kiểu dữ liệu do người dùng định nghĩa

- Hàm toán tử phải là thành viên của kiểu dữ liệu này, hoặc Phải có ít nhất một đối số thuộc kiểu dữ liệu tương ứng
- Chú ý khác biệt giữa:

aa+2

2+aa

Toán tử và Kiểu dữ liệu do người dùng định nghĩa – ví dụ với kiểu liệt kê

```
enum Day { sun, mon, tue, wed, thu, fri, sat };

Day& operator++(Day& d)
{
        return d = (sat==d) ? sun : static_cast<Day>(d+1);
}
```

Kiểu số phức (complex)

- Đọc về kiểu số phức trong phần 18.3, tài liệu “The C++ Programming Language 4ht Edition”

Chuyển kiểu

Có thể thực hiện bằng cách:

- Dùng constructor
  - Không xác định việc chuyển kiểu ngầm định từ kiểu do người dùng định nghĩa sang kiểu tích hợp sẵn
  - Không xác định được phép chuyển kiểu từ một lớp mới sang lớp đã được định nghĩa trước đó
- Dùng toán tử chuyển kiểu

Các toán tử chuyển kiểu

Hàm toán tử định nghĩa một phép chuyển đổi từ kiểu X sang kiểu T như sau:

X::operator T(),

trong đó **T** là tên kiểu.

Các toán tử chuyển kiểu – Ví dụ

Ví dụ: Định nghĩa kiểu **Tiny** là số nguyên không âm 6-bit có thể sử dụng lẫn với kiểu số nguyên thông thường trong các phép toán số học. **Tiny** gửi Bad_range để báo trường hợp phép toán gây lỗi tràn số (overflow) hoặc thiếu số (underflow):

```
class Tiny {
        char v;
        void assign(int i) { if (i&~077) throw Bad_range(); v=i; }
public:
        class Bad_range { };

        Tiny(int i) { assign(i); }
        Tiny& operator=(int i) { assign(i); return *this; }

        operator int() const { return v; }      // conversion to int function
};
```

```
int main()
{
        Tiny c1 = 2;
        Tiny c2 = 62;
        Tiny c3 = c2–c1;        // c3 = 60
        Tiny c4 = c3;           // no range check (not necessary)
        int i = c1+c2;          // i = 64

        c1 = c1+c2;             // range error: c1 can't be 64
        i = c3–64;              // i = -4
        c2 = c3–64;             // range error: c2 can't be -4
        c3 = c4;                // no range check (not necessary)
}
```

Các toán tử chuyển kiểu explicit

```
template <typename T, typename D = default_delete<T>>
class unique_ptr {
public:
        // ...
        explicit operator bool() const noexcept;        // does *this hold a pointer (that is not nullptr)?
        // ...
};
```

```
void use(unique_ptr<Record> p, unique_ptr<int> q)
{
        if (!p)                 // OK: we want this use
                throw Invalid_uninque_ptr{};

        bool b = p;         // error; suspicious use
        int x = p+q;        // error; we definitly don't want this
}
```

Tính mơ hồ (ambiguity)

Phép gán một giá trị của kiểu **V** vào một đối tượng thuộc kiểu lớp **X** là hợp lệ nếu có một toán tử gán **X::operator=(Z)** sao cho **V** là **Z** hoặc có một phép chuyển đổi duy nhất từ **V** thành **Z**.

Nếu một giá trị của kiểu chuyển đến có thể được khởi tạo bằng nhiều cách → không hợp lệ.

Do đó, không định nghĩa phép chuyển kiểu bằng cả hai cách: dùng hàm tạo constructor và dùng hàm toán tử chuyển đổi.

Tính mơ hồ (ambiguity) – Ví dụ 01

```
class X { /* ... */ X(int); X(const char*); };
class Y { /* ... */ Y(int); };
class Z { /* ... */ Z(X); };

X f(X);
Y f(Y);

Z g(Z);

void k1()
{
        f(1);                   // error: ambiguous f(X(1)) or f(Y(1))?
        f(X{1});                // OK
        f(Y{1});                // OK

        g("Mack");              // error: two user-defined conversions needed; g(Z{X{"Mack"}}) not tried
        g(X{"Doc"});            // OK: g(Z{X{"Doc"}})
        g(Z{"Suzy"});           // OK: g(Z{X{"Suzy"}})
}
```

Tính mơ hồ (ambiguity) – Ví dụ 02

```
class Quad {
public:
        Quad(double);
        // ...
};

Quad operator+(Quad,Quad);

void f(double a1, double a2)
{
        Quad r1 = a1+a2;          // double-precision floating-point add
        Quad r2 = Quad{a1}+a2;    // force quad arithmetic
}
```

Các toán tử đặc biệt

[] () –> ++ –– new delete

Phạm vi áp dụng viết chồng hàm

Các toán tử có vai trò quan trọng trong:

- Các container. Ví dụ: **vector**, **map**
- Các con trỏ đặc biệt. Ví dụ: **unique_ptr**, **shared_ptr**
- Các phép lặp

[]

Hàm toán tử operator[]() phải là hàm thành viên non-static.

```
struct Assoc {
	vector<pair<string,int>> vec;  // vector of {name,value} pairs

	const int& operator[] (const string&) const;
	int& operator[](const string&);
};
```

```
int& Assoc::operator[](const string& s)
	// search for s; return a reference to its value if found;
	// otherwise, make a new pair {s,0} and return a reference to its value
{
	for (auto x : vec)
		if (s == x.first) return x.second;

	vec.push_back({s,0});		// initial value: 0

	return vec.back().second;	// return last element
}
```

```
int main()	// count the occurrences of each word on input
{
	Assoc values;
	string buf;
	while (cin>>buf) ++values[buf];
	for (auto x : values.vec)
		cout << '{' << x.first << ',' << x.second << "}\n";
}
```

() - toán tử gọi hàm

- Đối tượng đóng vai trò hàm được gọi là đối tượng hàm

```
struct Action {
        int operator()(int);
        pair<int,int> operator()(int,int);
        double operator()(double);
        // ...
};

void f(Action act)
{
        int x = act(2);
        auto y = act(3,4);
        double z = act(2.3);
        // ...
};
```

() - toán tử gọi hàm – Ví dụ

```
class Add {
        complex val;
public:
        Add(complex c) :val{c} { }                          // save a value
        Add(double r, double i) :val{{r,i}} { }

        void operator()(complex& c) const { c += val; }     // add a value to argument
};
```

```
void h(vector<complex>& vec, list<complex>& lst, complex z)
{
        for_each(vec.begin(),vec.end(),Add{2,3});
        for_each(lst.begin(),lst.end(),Add{z});
}
```

```
template<typename Iter, typename Fct>
Fct for_each(Iter b, Iter e, Fct f)
{
        while (b != e) f(*b++);
        return f;
}
```

-> - Toán tử hủy tham chiếu

Toán tử hủy tham chiếu **->** (toán tử mũi tên), có thể được định nghĩa như là một toán tử hậu tố đơn phân.

Ví dụ:

```
class Ptr {
        // ...
        X* operator->();
};
```

Đối tượng của lớp **Ptr** có thể được sử dụng để truy xuất đến các thành viên của lớp **X**, tương tự như các dùng con trỏ.

Ví dụ:

```
void f(Ptr p)
{
        p->m = 7;        // (p.operator->())->m = 7
}
```

-> - Ví dụ

```
template<typename T>
class Disk_ptr {
        string identifier;
        T* in_core_address;
        // ...
public:
        Disk_ptr(const string& s) : identifier{s}, in_core_address{nullptr} { }
        ~Disk_ptr() { write_to_disk(in_core_address,identifier); }

        T* operator->()
        {
                if (in_core_address == nullptr)
                        in_core_address = read_from_disk(identifier);
                return in_core_address;
        }
};
```

```
struct Rec {
        string name;
        // ...
};

void update(const string& s)
{
        Disk_ptr<Rec> p {s};            // get Disk_ptr for s

        p->name = "Roscoe";             // update s; if necessary, first retrieve from disk
        // ...
}                                       // p's destructor writes back to disk
```

++ và --

```
void f1(X a)            // traditional use
{
        X v[200];
        X* p = &v[0];
        p--;
        *p = a;     // oops: p out of range, uncaught
        ++p;
        *p = a;     // OK
}
```

```
void f2(Ptr<X> a)           // checked
{
        X v[200];
        Ptr<X> p(&v[0],v);
        p--;
        *p = a;     // run-time error: p out of range
        ++p;
        *p = a;     // OK
}
```

++ và --

Định nghĩa lại các phép ++ và -- trong cả hai trường hợp dùng: hậu tố và tiền tố.

```
template<typename T>
class Ptr {
        T* ptr;
        T* array;
        int sz;
public:
        template<int N>
                Ptr(T* p, T(&a)[N]);        // bind to array a, sz==N, initial value p
        Ptr(T* p, T* a, int s);             // bind to array a of size s, initial value p
        Ptr(T* p);                          // bind to single object, sz==0, initial value p

        Ptr& operator++();                  // prefix
        Ptr operator++(int);                // postfix

        Ptr& operator--();                  // prefix
        Ptr operator--(int);                // postfix

        T& operator*();                     // prefix
};
```

++ và --

Định nghĩa lại các phép ++ và -- trong cả hai trường hợp dùng: hậu tố và tiền tố.

Phép cộng-tiền tố có thể trả về một tham chiếu đến đối tượng của nó.

Phép cộng-hậu tố phải tạo ra một đối tượng mới rồi sau đó mới trả về đối tượng này.

Ví dụ:

```
template<typename T>
Ptr& Ptr<T>::operator++()          // return the current object after incrementing
{
        // ... check that ptr+1 can be pointed to ...
        return *++ptr;
}
```

```
template<typename T>
Ptr Ptr<T>::operator++(int)        // increment and return a Ptr with the old value
{
        // ... check that ptr+1 can be pointed to ...
        Ptr<T> old {ptr,array,sz};
        ++ptr;
        return old;
}
```

```
void f3(T a)            // checked
{
        T v[200];
        Ptr<T> p(&v[0],v,200);
        p.operator--(0);            // suffix: p--
        p.operator*() = a;          // run-time error: p out of range
        p.operator++();             // prefix: ++p
        p.operator*() = a;          // OK
}
```

new và delete

- Toán tử **new** gọi operator **new()** để cấp phát bộ nhớ
- Toán tử **delete** gọi operator **delete()** để giải phóng phần bộ nhớ đã cấp phát

- Khai báo:

```
void* operator new(size_t);              // use for individual object
void* operator new[](size_t);            // use for array
void operator delete(void*, size_t);     // use for individual object
void operator delete[](void*, size_t);   // use for array
```

new và delete – Ví dụ

```
class Employee {
public:
        // ...

        void* operator new(size_t);
        void operator delete(void*, size_t);

        void* operator new[](size_t);
        void operator delete[](void*, size_t);
};
```

```
void* Employee::operator new(size_t s)
{
        // allocate s bytes of memory and return a pointer to it
}

void Employee::operator delete(void* p, size_t s)
{
        if (p) {        // delete only if p!=0; see §11.2, §11.2.3
                // assume p points to s bytes of memory allocated by Employee::operator new()
                // and free that memory for reuse
        }
}
```

Các giá trị do người dùng định nghĩa

```
123          // int
1.2          // double
1.2F         // float
'a'          // char
1ULL         // unsigned long long
0xD0         // hexadecimal unsigned
"as"         // C-style string (const char[3])
```

```
"Hi!"s                          // string, not “zero-terminated array of char”
1.2i                            // imaginary
101010111000101b                // binary
123s                            // seconds
123.56km                        // not miles! (units)
1234567890123456789012345678901234567890x   // extended-precision
```

Các giá trị do người dùng định nghĩa

Toán tử định trị (literal operator) ánh xạ một giá trị có kèm theo một hậu tố vào kiểu dữ liệu muốn dùng.

Ví dụ:

```
constexpr complex<double> operator"" i(long double d)      // imaginary literal
{
        return {0,d};     // complex is a literal type
}

std::string operator"" s(const char* p, size_t n)      // std::string literal
{
        return string{p,n};   // requires free-store allocation
}
```

```
template<typename T> void f(const T&);

void g()
{
        f("Hello");        // pass pointer to char*
        f("Hello"s);       // pass (five-character) string object
        f("Hello\n"s);     // pass (six-character) string object

        auto z = 2+1i;     // complex{2,1}
}
```

Định nghĩa giá trị với hậu tố

- Giá trị số nguyên: dùng toán tử định trị với đối số **unsigned long long** hoặc **const char***, hoặc dùng toán tử định trị mẫu, ví dụ **123m** hoặc **12345678901234567890X**

- Giá trị số thực: dùng toán tử định trị với đối số long double hoặc const char*, hoặc dùng toán tử định trị mẫu, ví dụ **12345678901234567890.976543210x** hoặc **3.99s**

- Giá trị chuỗi: dùng toán tử định trị nhận đối số là (**const char*, size_t**), ví dụ **“string”s**, **R”Foo\bar)”_path**

- Giá trị ký tự: dùng toán tử định trị nhận đối số là ký tự thuộc kiểu **char**, **wchar_t**, **char16_t**. Ví dụ **‘f’_runic**, **u’BEEP’_w**.

Ví dụ

Hỗ trợ nhập số nguyên “lớn” với chiều dài bất kỳ.

```
Bignum operator"" x(const char* p)
{
        return Bignum(p);
}

void f(Bignum);

f(12345678901234567890123456789012345678901234 5x);
```

Ví dụ

```
string operator"" s(const char* p, size_t n);

string s12 = "one two"s;        // calls operator ""("one two",7)
string s22 = "two\ntwo"s;       // calls operator ""("two\ntwo",7)
string sxx = R"(two\ntwo)"s;    // calls operator ""("two\\ntwo",8)
```

Lớp String và các toán tử

Lớp String

```
class String {
public:
        String();                                   // default constructor: x{""}

        explicit String(const char* p);             // constructor from C-style string: x{"Euler"}

        String(const String&);                      // copy constructor
        String& operator=(const String&);           // copy assignment


        String(String&& x);                         // move constructor
        String& operator=(String&& x);              // move assignment

        ~String() { if (short_max<sz) delete[] ptr; }   // destructor

        // ...
};
```

Truy xuất đến các ký tự bằng []

Khai báo

```
class String {
public:
    // ...

    char& operator[](int n) { return ptr[n]; }          // unchecked element access
    char operator[](int n) const { return ptr[n]; }

    char& at(int n) { check(n); return ptr[n]; }        // range-checked element access
    char at(int n) const { check(n); return ptr[n]; }

    String& operator+=(char c);                         // add c at end

    const char* c_str() { return ptr; }                 // C-style string access
    const char* c_str() const { return ptr; }

    int size() const { return sz; }                     // number of elements
    int capacity() const                                // elements plus available space
        { return (sz<=short_max) ? short_max : sz+space; }

    // ...
};
```

Truy xuất đến các ký tự bằng [] - Ví dụ

```
int hash(const String& s)
{
        int h {s[0]};
        for (int i {1}; i!=s.size(); i++) h ^= s[i]>>1;        // unchecked access to s
        return h;
}
```

Đại diện cho String

- Dễ dàng chuyển một chuỗi kiểu C-style sang kiểu **String** và cho phép dễ truy xuất đến các ký tự trong một **String** tương tự như đối với chuỗi C-style
- Giảm thiểu việc sử dụng bộ nhớ
- Hỗ trợ thêm (add) ký tự ở phía cuối của một **String**

Đại diện cho String

```
class String {
/*
        A simple string that implements the short string optimization

        size()==sz is the number of elements
        if size()<= short_max, the characters are held in the String object itself;
        otherwise the free store is used.

        ptr points to the start of the character sequence
        the character sequence is kept zero-terminated: ptr[size()]==0;
        this allows us to use C library string functions and to easily return a C-style string: c_str()

        To allow efficient addition of characters at end, String grows by doubling its allocation;
        capacity() is the amount of space available for characters
        (excluding the terminating 0): sz+space
*/
public:
        // ...
private:
        static const int short_max = 15;
        int sz;                         // number of characters
        char* ptr;
        union {
                int space;              // unused allocated space
                char ch[short_max+1];   // leave space for terminating 0
        };

        void check(int n) const         // range check
        {
                if (n<0 || sz<=n)
                        throw std::out_of_range("String::at()");
        }

        // ancillary member functions:
        void copy_from(const String& x);
        void move_from(String& x);
};
```

Hàm friend

Một khai báo hàm thành viên thông thường có 3 đặc điểm sau:

1. Hàm có thể truy xuất đến phần riêng (private) của khai báo lớp
2. Hàm nằm trong phạm vi (scope) của lớp
3. Hàm phải được gọi theo đối tượng của lớp (có con trỏ **this**)

- Nếu khai báo hàm là **static**, thì hàm chỉ có (1) và (2)

- Nếu khai báo một hàm không phải thành viên là **friend**, hàm này chỉ có (1) i.e., hàm **friend** được cấp quyền truy xuất đến phần cài đặt của lớp tương đương như vai trò hàm thành viên của lớp nhưng lại độc lập với lớp này.

Ví dụ:

```
constexpr rc_max {4};     // row and column size

class Matrix;

class Vector {
        float v[rc_max];
        // ...
        friend Vector operator*(const Matrix&, const Vector&);
};
```

```
class Matrix {
        Vector v[rc_max];
        // ...
        friend Vector operator*(const Matrix&, const Vector&);
};
```

```
Vector operator*(const Matrix& m, const Vector& v)
{
        Vector r;
        for (int i = 0; i!=rc_max; i++) {          // r[i] = m[i] * v;
                r.v[i] = 0;
                for (int j = 0; j!=rc_max; j++)
                        r.v[i] += m.v[i].v[j] * v.v[j];
        }
        return r;
}
```

Hàm friend – các đặc tính

- Khai báo friend có thể đặt ở phần private hoặc phần public
- Hàm friend được khai báo tường minh trong lớp "bạn" của hàm
- Hàm friend của một lớp có thể là friend của một lớp khác.
  Ví dụ:

```
class List_iterator {
        // ...
        int* next();
};

class List {
        friend int* List_iterator::next();
        // ...
};
```

Khai báo friend cho một lớp

- Cấp quyền truy xuất đến tất cả các hàm của lớp đó.

```
class List {
        friend class List_iterator;
        // ...
};
```

- Đối số mẫu thành **friend**

```
template<typename T>
class X {
        friend T;
        friend class T;  // redundant "class"
        // ...
};
```

Ví dụ

```
class C1 { };     // will become friend of N::C
void f1();        // will become friend of N::C

namespace N {
    class C2 { };         // will become friend of C
    void f2() { }         // will become friend of C

    class C {
        int x;
    public:
        friend class C1;      // OK (previously defined)
        friend void f1();

        friend class C3;      // OK (defined in enclosing namespace)
        friend void f3();
        friend class C4;      // First declared in N and assumed to be in N
        friend void f4();
    };

    class C3 {};              // friend of C
    void f3() {  C x; x.x = 1; }  // OK: friend of C
} // namespace N

class C4 { };                 // not friend of N::C
void f4() { N::C x; x.x = 1; }  // error: x is private and f4() is not a friend of N::C
```

Các hàm friend và hàm thành viên

Chọn cách nào để viết các toán tử?

Quy tắc:

- Giảm thiểu số lượng các hàm có truy xuất đến đại diện một lớp
- Tạo một tập hợp các hàm truy xuất phù hợp

Các hàm friend và hàm thành viên

Ví dụ: minh họa các cách viết hàm toán tử với khai báo friend và khai báo thành viên.

```
class X {
        // ...
        X(int);

        int m1();                       // member
        int m2() const;

        friend int f1(X&);              // friend, not member
        friend int f2(const X&);
        friend int f3(X);
};
```

Các hàm friend và hàm thành viên

- Hàm thành viên chỉ được gọi khi áp dụng cho các đối tượng thuộc lớp chứa hàm. Không áp dụng cách chuyển kiểu do người dùng định nghĩa cho toán hạng chính của phép tham chiếu **.** Và **->** .

Ví dụ:

```
void g()
{
        99.m1();   // error: X(99).m1() not tried
        99.m2();   // error: X(99).m2() not tried
}
```

- Chuyển kiểu không tường minh không được áp dụng cho các đối số tham chiếu non-**const**.
  → Toán tử thay đổi trạng thái của đối tượng một lớp là thành viên hoặc là hàm nhận đối số tham chiếu non-**const** (hoặc đối số là con trỏ tới non-**const**)

```
void h()
{
        f1(99);     // error: f1(X(99)) not tried: non-const X& argument
        f2(99);     // OK: f2(X(99)); const X& argument
        f3(99);     // OK: f3(X(99)); X argument
}
```

Kết thúc phần này